Ghulam Ishaq Khan Institute of Engineering Sciences and Technology

Faculty of Computer Engineering

AXI-MM Based Cache Implementation

Group Members:

M. Abdullah Saeed	**Reg. No. 2022329**
M. Abdullah Zafar	**Reg. No. 2022328**
Aqib Muhammad	**Reg. No. 2022103**
Supervisor:	**Dr. Waqar Ahmad**
Co-Supervisor:	**Dr. Fahad Bin Muslim**
Session:	**2025–2026**

Faculty of Computer Engineering

Ghulam Ishaq Khan Institute of Engineering Sciences and Technology

April 2026

CERTIFICATE OF APPROVAL

It is certified that the work presented in this Final Year Project report, titled **"AXI-MM Based Cache Implementation,"** was performed by **M. Abdullah Saeed (Reg. No. 2022329), M. Abdullah Zafar (Reg. No. 2022328), and Aqib Muhammad (Reg. No. 2022103)** under the supervision of **Dr. Waqar Ahmad**, Faculty of Computer Engineering, Ghulam Ishaq Khan Institute of Engineering Sciences and Technology. The work satisfies the requirements for the award of the degree of Bachelor of Science in Computer Engineering and is approved for submission.

_____

Dr. Waqar Ahmad
Advisor
Faculty of Computer Engineering
GIK Institute

_____

Dr. Fahad Bin Muslim
Co-Advisor
Faculty of Computer Engineering
GIK Institute

_____

Prof. Dr. Qadeer Ul Hassan
Dean, Faculty of Computer Engineering
GIK Institute

SDGs INCLUSION

This project aligns with the following United Nations Sustainable Development Goals (SDGs):

SDG 9 — Industry, Innovation and Infrastructure

The AXI-MM Based Cache subsystem directly supports Goal 9 by advancing hardware infrastructure for high-performance, energy-efficient computing systems. Designing a formally verified, 4-way set-associative cache with an AXI4-compliant interface and a non-blocking miss policy demonstrates the application of modern engineering innovation to build resilient and efficient digital infrastructure, which is a prerequisite for industrial-grade embedded and SoC platforms.

Dr. Waqar Ahmad	**Dr. Fahad Bin Muslim**
Advisor	Co-Advisor

ABSTRACT

This report presents the design and functional verification of an AXI4 memory-mapped 4-way set-associative cache subsystem implemented in SystemVerilog. The cache is 4 KB in capacity, organised as 128 sets of 4 ways with a 64-bit (8-byte) cache line. It employs a write-through, no-write-allocate policy with Pseudo-Least-Recently-Used (PLRU) replacement and a 4-entry Miss Status Holding Register (MSHR) to track outstanding memory fetches. Write traffic is decoupled from the CPU critical path through an 8-entry write buffer that drains to main memory asynchronously.

The subsystem implements a *non-blocking* miss policy: on a cold read miss the arbiter dequeues the request immediately after MSHR allocation, without stalling the request pipeline. The cache continues to serve subsequent requests while the memory fetch is outstanding. Fill responses are delivered to the CPU through a depth-4 fill response FIFO. A separate WAIT_PENDING state handles the narrower case where a second request targets an address whose fill is already in flight (a pending-hit stall), ensuring correct ordering without broader pipeline blocking.

The entire subsystem—including tag store, status store, data store, PLRU state store, hit detector, control FSM, MSHR, write buffer, read/write FIFOs, round-robin arbiter, and a behavioural main-memory model—is integrated in a single top-level AXI4 slave module. Simulation was performed using Cadence Xcelium 25.03. Ten directed test cases covering cold misses, cache hits, write-through behaviour, RAW forwarding, write hits, hit-under-miss, concurrent misses, and pending-hit stall were executed; all 27 assertions passed with zero failures. Functional coverage collected via SystemVerilog covergroups confirms 100% exercise of fill-completion events, non-blocking miss completions, and read/write $\times$ hit/miss cross-coverage.

ACKNOWLEDGEMENTS

The authors thank the Faculty of Computer Engineering at GIK Institute for providing access to EDA tools and laboratory resources. Special thanks to Dr. Waqar Ahmad and Dr. Fahad Bin Muslim for their guidance and technical mentorship throughout the project. The authors also acknowledge Cadence Design Systems for providing access to the Xcelium Logic Simulator through the EDA Playground online platform.

Contents

Abstract **i**

Acknowledgements **ii**

1 Introduction **1**

1.1 Motivation 1
1.2 Objectives 1
1.3 Contributions 2
1.4 Report Organisation 3

2 Background **4**

2.1 Cache Memory Fundamentals 4
2.2 Set-Associative Organisation 4
2.3 Write Policies 5
2.4 Replacement Policy — Pseudo-LRU 5
2.5 Miss Status Holding Register 6
2.6 Write Buffer 7
2.7 Non-Blocking Cache Architecture 7
2.8 AXI4 Protocol 8
2.9 Related Work 9

3 System Architecture **10**

3.1 Top-Level View 10
3.2 Cache Parameters 10
3.3 Address Mapping 11
3.4 Request Pipeline 12

3.5 Non-Blocking Miss Flow 12
3.6 Fill-Response Delivery Path 13

4 Module-Level Design 15
4.1 Tag Store 15
4.2 Status Bits Store 15
4.3 Hit Detector 16
4.4 PLRU Store 16
4.4.1 Tree Structure 16
4.4.2 Update Rules 17
4.5 Data Store 17
4.6 Control FSM 18
4.6.1 States and Non-Blocking Property 18
4.6.2 IDLE State Behaviour 18
4.6.3 WAIT_PENDING State 19
4.6.4 STALL_FULL State 20
4.6.5 mm_rcvd Block 20
4.7 Round-Robin Arbiter 21
4.7.1 Design 21
4.7.2 FIFO Re-Presentation Guard 21
4.8 MSHR 22
4.8.1 Entry Lifecycle 22
4.8.2 AXI Read Master 22
4.8.3 Zombie State 22
4.8.4 Multi-Entry Operation 23
4.9 Write Buffer 23
4.9.1 Structure 23
4.9.2 AXI Write Master FSM 23
4.9.3 RAW Forwarding 23
4.10 Fill Response FIFO 24
4.11 Main Memory Model 24
4.12 Known Limitations 24

5 Verification **26**
5.1 Testbench Architecture 26
5.2 Test Cases 26
5.3 Test Case Descriptions 26
5.3.1 TC1 — Read Miss Followed by Hit 26
5.3.2 TC2 — Write Then Read 27
5.3.3 TC3 — PLRU Victim Coverage 28
5.3.4 TC4 — Consecutive Writes 28
5.3.5 TC5 — Mixed Read/Write 28
5.3.6 TC6 — RAW Forwarding 28
5.3.7 TC7 — Write Hit 29
5.3.8 TC8 — Hit Under Miss 29
5.3.9 TC9 — Concurrent Misses 29
5.3.10 TC10 — Pending-Hit Stall 30
5.4 Functional Coverage 31
5.4.1 Coverage Groups 31
5.4.2 Non-Blocking Covergroup 31
5.4.3 Tool Setup 32

6 Results and Discussion **33**
6.1 Simulation Results 33
6.2 Simulation Waveforms 33
6.2.1 TC1 — Non-Blocking Read Miss 33
6.2.2 TC1 — Subsequent Cache Hit 34
6.2.3 TC8 — Hit Under Miss 34
6.2.4 TC9 — Concurrent Misses 35
6.2.5 TC10 — Pending-Hit Stall 36
6.3 Functional Coverage Results 36
6.4 Discussion 38
6.4.1 Non-Blocking Property 38
6.4.2 Covered Scenarios 39
6.4.3 Uncovered Coverage and Implications 39

6.4.4 Performance Implications of Non-Blocking Operation 39

7 Conclusion and Future Work 40

7.1 Conclusion 40

7.2 Future Work 40

A Module Hierarchy 43

B Simulation Console Output 44

C Key RTL Snippets 46

List of Figures

3.1 Cache subsystem top-level architecture 10
3.2 32-bit address decomposition: 22-bit tag, 7-bit set index, 3-bit offset 11
3.3 Serialised request pipeline with arbiter back-pressure 13
3.4 R-channel response priority multiplexer 14

4.1 3-bit PLRU binary decision tree for a 4-way set 17
4.2 3-state non-blocking control FSM — state transition diagram . 18
4.3 MSHR entry state transitions 22
4.4 Write buffer drain state machine 23

6.1 TC1: Non-blocking cold read miss — `mshr_alloc_en` dequeues the request at miss time; `stall=0` throughout; fill data delivered via fill FIFO on `mm_rcvd` 34
6.2 TC1: Cache hit — `hd_hit` asserts immediately; `resp_en` fires; data returned via `rresp` register with single-cycle latency . . . 35
6.3 TC8: Hit under miss — AR2 (hit) is served while AR1 fill is in flight; `stall=0` throughout; both responses correct 35
6.4 TC9: Concurrent misses — two MSHR entries allocated on consecutive cycles; both fills complete and deliver responses via the fill FIFO; `stall=0` during both allocations 36
6.5 TC10: Pending-hit stall — AR2 to an in-flight line asserts `stall=1`; on `mm_rcvd`, `pending_resolved` fires and AR2 receives fill data via `rresp` register 37
6.6 Functional coverage per covergroup — blue: covered, red: not covered 38

List of Tables

2.1 AXI4 channel summary 8

3.1 Cache design parameters 11

4.1 Line status encoding 15
4.2 PLRU bit update rules 17

5.1 Directed test case summary 27
5.2 Covergroup definitions 31

6.1 Simulation pass/fail summary 33
6.2 Functional coverage detailed breakdown 37

A.1 RTL module hierarchy 43

CHAPTER I
Introduction

1.1 Motivation

Modern processors execute instructions several times faster than main memory can supply operands. Even a DDR5 DRAM with a raw bandwidth of 60 GB/s imposes latencies of 60–80 clock cycles on a 3 GHz processor, creating a severe *memory wall* that limits instruction-level parallelism. Cache memories—small, fast SRAM structures that exploit spatial and temporal locality—bridge this gap by serving the majority of memory references with single-cycle latency.

The memory wall problem is especially acute in pipelined and out-of-order processors, where a single cache miss can drain the instruction window and halt forward progress across multiple execution units. The classic solution is to combine a cache with a lockup-free (non-blocking) miss policy: on a cache miss the pipeline is not frozen; instead the miss is recorded in a Miss Status Holding Register (MSHR) and execution continues. If a subsequent request hits in the cache, it is served normally. If it targets the same in-flight line, it is briefly stalled only until the fill arrives. This hit-under-miss capability converts long-latency memory stalls from *blocking* events into overlapped background activities.

As system-on-chip (SoC) designs increasingly adopt the Advanced eXtensible Interface (AXI4) as the standard on-chip interconnect, a cache that exposes an AXI4 slave interface becomes a first-class building block compatible with any AXI4 master, from soft-core processors to DMA engines. This project designs, implements, and verifies exactly such a component: a non-blocking, 4-way set-associative cache subsystem with an AXI4 interface, fully simulated and coverage-verified on Cadence Xcelium.

1.2 Objectives

The project has six primary objectives:

1. Design a 4-way set-associative cache with PLRU replacement and write-through, no-write-allocate policy.
2. Implement a fully AXI4-compliant slave interface accepting independent read (AR/R) and write (AW/W/B) transactions.
3. Integrate a 4-entry MSHR to track outstanding read-miss fetches and an 8-entry write buffer to decouple write traffic from the CPU critical path.
4. Implement a non-blocking miss policy so that cold misses dequeue the request pipeline immediately and fill completions are returned via a dedicated fill-response FIFO.
5. Develop a directed SystemVerilog testbench covering all major transaction scenarios including hit-under-miss, concurrent misses, and pending-hit stall.
6. Measure and report functional coverage using SystemVerilog covergroups on Cadence Xcelium.

1.3 Contributions

The specific technical contributions of this project are:

- A complete RTL implementation of a non-blocking 4-way set-associative cache in SystemVerilog, totalling 13 modules and approximately 1 800 lines of synthesisable RTL.
- A three-state control FSM (IDLE/WAIT_PENDING/STALL_FULL) in which cold misses are *never blocking*: the arbiter dequeues the missed request on the same cycle as MSHR allocation.
- A depth-4 fill-response FIFO that decouples fill-completion delivery from the synchronous request pipeline, allowing up to four fills to complete independently.

- A correction to the round-robin arbiter's FIFO re-presentation guard, preventing spurious re-consumption of already-popped FIFO entries when out_ready and a new FIFO entry coincide.
- A ten-test directed testbench with 27 assertions verified at zero failures, plus seven SystemVerilog covergroups measuring functional completeness.

1.4 Report Organisation

Chapter 2 reviews cache architecture fundamentals, the MSHR concept, and non-blocking cache theory. Chapter 3 presents the overall system architecture. Chapter 4 describes each sub-module in detail, including the fill-response path. Chapter 5 covers the testbench, all ten test cases, and the coverage methodology. Chapter 6 presents simulation results, annotated waveforms, and coverage analysis. Chapter 7 concludes and identifies future work.

CHAPTER II

Background

2.1 Cache Memory Fundamentals

A cache memory is a small, fast memory that transparently stores copies of frequently accessed main-memory locations. On every memory access the processor first checks the cache; if the requested address is present (*cache hit*), data is returned with low latency. If not (*cache miss*), the block containing the requested data is fetched from main memory and installed in the cache.

Cache effectiveness is measured by the *hit rate*—the fraction of accesses served from the cache. A 95% hit rate, for example, means only 5% of accesses incur main-memory latency. Because cache accesses are typically $10\times$ to $100\times$ faster than DRAM, even modest hit rates produce dramatic average memory access time reductions.

The average memory access time (AMAT) formula makes this quantitative:

$$\text{AMAT} = t_{\text{hit}} + (1-h)\cdot t_{\text{miss}}$$

where h is the hit rate, t_{hit} is cache access latency, and t_{miss} is the main-memory penalty. For a 3-GHz processor with $t_{\text{hit}} = 1\,\text{cycle}$, $t_{\text{miss}} = 60\,\text{cycles}$, and $h = 0.95$, $\text{AMAT} = 1 + 0.05 \times 60 = 4\,\text{cycles}$ versus $60\,\text{cycles}$ without a cache—a $15\times$ improvement.

2.2 Set-Associative Organisation

A direct-mapped cache maps each memory address to exactly one cache line, causing *conflict misses* when two frequently used addresses compete for the same line. A fully-associative cache eliminates this by allowing any line to occupy any way, but hardware cost grows with associativity. *Set-associative* caches balance the two extremes: the address is used to select a *set* (a small group of ways), and the line may be placed in any way within that set. Four-way

associativity is the most common choice in L1 data caches, providing most of the hit-rate benefit of full associativity at a fraction of the cost.

For a 4-way set-associative cache with S sets and B-byte cache lines, the address is partitioned as follows: $\log_2 B$ bits form the byte offset, $\log_2 S$ bits form the set index, and the remaining bits form the tag. Two addresses conflict (occupy the same set) only if their set index fields are identical. With $S = 128$ and $B = 8$, the conflict rate is far lower than a direct-mapped cache of equal capacity.

2.3 Write Policies

Two dimensions characterise write behaviour:

Write-through vs. write-back. Write-through immediately propagates every write to main memory, ensuring coherence at the cost of higher write bandwidth. Write-back defers the propagation until the line is evicted, reducing bandwidth but requiring a dirty bit and eviction logic.

Write-allocate vs. no-write-allocate. Write-allocate fetches the line on a write miss before applying the write, keeping the cache warm for subsequent reads. No-write-allocate simply forwards the write to memory without populating the cache.

Write-through paired with no-write-allocate is the simpler policy and is well-suited to designs where the write buffer effectively hides the write latency. This project uses write-through, no-write-allocate.

2.4 Replacement Policy — Pseudo-LRU

When all ways in a set are occupied and a new line must be installed, the cache must evict an existing line. Least-Recently-Used (LRU) is optimal in theory but requires $\log_2(W!)$ bits per set and a comparator network. *Pseudo-LRU* (PLRU) approximates LRU with only $W-1$ bits per set using a binary decision tree. For $W=4$ ways, three bits suffice:

- Bit 0 (root): 0 selects the left subtree (ways 0 and 1), 1 selects the right subtree (ways 2 and 3).

- Bit 1 (left subtree): 0 selects way 1, 1 selects way 0.
- Bit 2 (right subtree): 0 selects way 3, 1 selects way 2.

On every access, the three bits are updated to point *away from* the accessed way, ensuring the most recently used way is the last to be evicted. The approximation error compared to true LRU is negligible for workloads with strong temporal locality.

2.5 Miss Status Holding Register

A Miss Status Holding Register (MSHR) [1] was introduced by Kroft in 1981 to enable *lockup-free* cache operation. The fundamental observation is that, once a miss has been recorded and the memory request issued, the cache pipeline contains all information needed to complete the transaction when the data arrives. There is no need to *lock up* the pipeline waiting for it.

Each MSHR entry stores:

- The target address (tag and set), used to reconstruct the AXI AR address and route the fill to the correct line.
- The target way, determined by the PLRU victim selection at miss time.
- The transaction ID of the originating CPU request, used to generate the correct AXI R response.

When a fill completes, the MSHR entry is freed and a *fill notification* (`mm_rcvd`) is generated. This drives the status-store update (PENDING $\rightarrow$ VALID), the data-store write, and the delivery of the R response to the CPU.

The MSHR also prevents *duplicate miss requests*: if a second access to the same address arrives while the first fill is outstanding (a *pending hit*), the second request is stalled until the fill completes, after which both receive the same fill data. Without an MSHR this scenario would result in two independent main-memory requests for the same cache line, wasting bandwidth and risking write-after-read hazards.

2.6 Write Buffer

A write buffer is a FIFO that absorbs CPU writes and forwards them to main memory in the background. It decouples write latency from the CPU critical path and enables read-after-write (RAW) forwarding: if a subsequent read address matches a live write buffer entry, the buffered data is returned immediately without a main-memory access.

Write buffering is particularly beneficial for write-through caches, where every write would otherwise incur a full main-memory round trip. By absorbing bursts of writes into the FIFO and draining them when the memory bus is idle, the write buffer hides write latency without changing the cache's write policy.

2.7 Non-Blocking Cache Architecture

A *blocking* (or *lockup*) cache stalls the entire request pipeline while a fill is in progress. No new request can be accepted until the fill completes and the missed address is re-probed. This policy is simple to implement but severely limits throughput under workloads with moderate miss rates.

A *non-blocking* (or *lockup-free*) cache, in contrast, continues to accept and process new requests while a fill is outstanding. The key insight is that most subsequent requests will not target the in-flight address—they will hit in valid lines, be served normally, and never even encounter the outstanding fill. Only the narrow case of a second access to the *same* in-flight address (a pending hit) requires special handling.

Non-blocking operation requires the following hardware additions beyond a basic blocking cache:

1. **MSHR:** to record the in-flight miss so the pipeline knows which address is outstanding.

2. **Pending-status tracking:** a per-line status bit indicating that the line is allocated but not yet filled. The hit detector distinguishes VALID hits from PENDING hits.

3. **Fill-response delivery path:** when a fill completes asynchronously, the response must be delivered to the original requester even though the request has already been dequeued. This design uses a depth-4 fill-response FIFO.

4. **Pending-hit stall mechanism:** when a pending hit is detected, the arbiter must hold the request until the fill arrives, then deliver the fill data as the response.

The throughput advantage of non-blocking operation is most pronounced under high miss rates. If the main-memory latency is L cycles and the miss rate is m, a blocking cache wastes $m \cdot L$ cycles per request in stalls. A non-blocking cache can overlap these stalls with useful hits, reducing effective stall cycles to those caused only by pending-hit conflicts—a much rarer event.

2.8 AXI4 Protocol

AXI4 (AMBA Advanced eXtensible Interface 4) is an ARM-defined protocol for high-bandwidth, low-latency on-chip communication [2]. It separates read and write channels, each with independent flow control via `valid`/`ready` handshake pairs. The five channels are:

Table 2.1: AXI4 channel summary

Channel	Signals	Direction	Purpose
AR	`ARADDR, ARVALID, ARREADY`	M→S	Read address
R	`RDATA, RVALID, RREADY`	S→M	Read data
AW	`AWADDR, AWVALID, AWREADY`	M→S	Write address
W	`WDATA, WSTRB, WVALID, WREADY`	M→S	Write data
B	`BRESP, BVALID, BREADY`	S→M	Write response

A key property of AXI4 is that the AW and W channels are independent; the write data may arrive before, after, or simultaneously with the write address.

This project imposes the simplification that both must be presented in the same clock cycle (see Section 4.12).

AXI4 IDs allow multiple outstanding transactions: each AR (or AW) request carries an ID, and the slave's R (or B) response carries the same ID so the master can match responses to requests. This project uses 4-bit IDs, supporting up to 16 distinct outstanding transaction identifiers. The MSHR uses the entry index (0–3) as the AXI ID when issuing reads to main memory, allowing the R response to be matched back to the correct MSHR entry via `RID[1:0]`.

2.9 Related Work

The lockup-free cache concept was introduced by Kroft [1] in the context of scalar processors. Subsequent work by Tuck et al. extended MSHR design to superscalar out-of-order processors, where multiple MSHRs must coexist with reorder buffers [7]. Sorin et al. [8] address the verification challenge of lockup-free caches, noting that the interaction between fill completion, pipeline restart, and ordering constraints introduces subtle bugs that are difficult to catch without directed tests for the pending-hit corner case.

The design presented in this report most closely resembles the single-level, write-through variant described by Hennessy and Patterson [3] for simple pipeline processors, extended with a non-blocking miss path and an AXI4 interface. Unlike fully pipelined L1 caches in commercial processors, the design serialises requests through a single pipeline stage, making verification tractable while preserving the essential lockup-free property.

CHAPTER III

System Architecture

3.1 Top-Level View

Figure 3.1 shows the top-level block diagram of the cache subsystem. The CPU-facing AXI4 slave interface is split into a read path and a write path that converge through a round-robin arbiter onto a single serialised request pipeline.

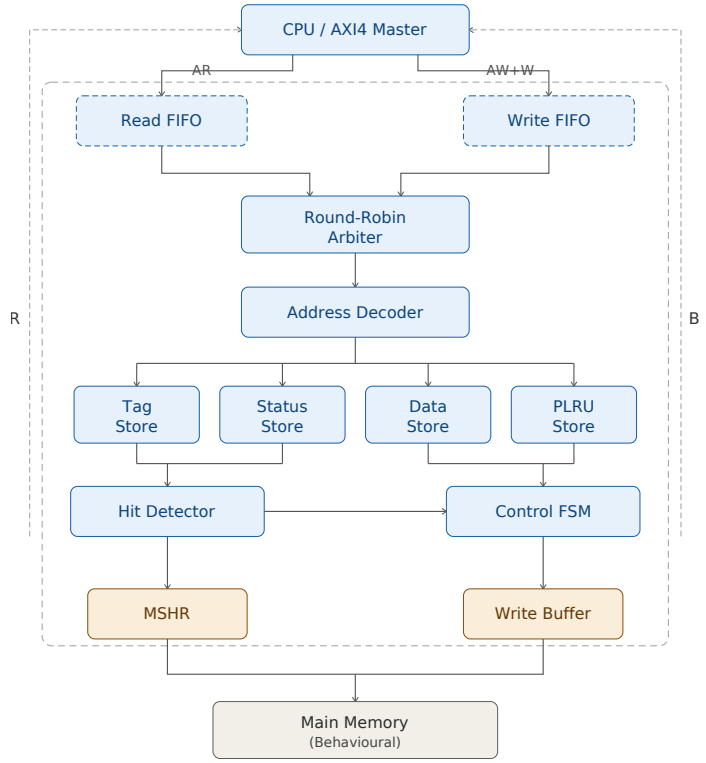

Figure 3.1: Cache subsystem top-level architecture

3.2 Cache Parameters

Table 3.1 summarises all design parameters as implemented in RTL.

Table 3.1: Cache design parameters

Parameter	Value	Notes
Total capacity	4 KB	$128 \times 4 \times 8$ bytes
Number of sets	128	7-bit set index
Associativity	4 ways	
Cache-line width	64 bits	Single AXI beat
Replacement policy	PLRU	3-bit binary tree per set
Write policy	Write-through, no-write-allocate	
Miss policy	Non-blocking	Cold miss dequeues immediately
AXI address width	32 bits	
AXI data width	64 bits	
AXI ID width	4 bits	
MSHR entries	4	Outstanding read misses
Fill response FIFO	4 entries	Matches MSHR capacity
Write buffer depth	8 entries	AXI FIFO
Read/write FIFO depth	8 entries	Per channel
Simulation tool	Cadence Xcelium 25.03	

3.3 Address Mapping

Each 32-bit AXI address is decomposed into three fields as shown in Figure 3.2. The 3-bit block offset addresses the 8-byte (64-bit) word within a cache line. The 7-bit set index selects one of 128 sets. The remaining 22 high-order bits form the tag compared against the tag store.

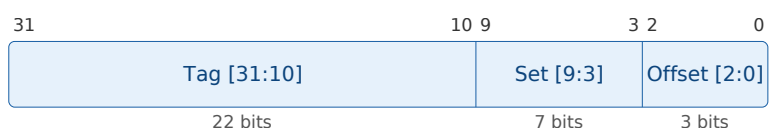

Figure 3.2: 32-bit address decomposition: 22-bit tag, 7-bit set index, 3-bit offset

The effective cache capacity follows directly:

$$\text{Capacity} = \underbrace{128}_{\text{sets}} \times \underbrace{4}_{\text{ways}} \times \underbrace{8\,\text{B}}_{\text{line}} = 4\,\text{KB}$$

In RTL, the decoder extracts these fields from the serialised arbiter

output as:

```
req_tag = req_addr[31:10]    (22 bits)
req_set = req_addr[9:3]    (7 bits)
offset = req_addr[2:0]    (3 bits, used only for byte-enable generation)
```

3.4 Request Pipeline

Figure 3.3 illustrates the serialised request pipeline. AR transactions are buffered in the read FIFO; AW+W transactions (which must arrive simultaneously) are buffered in the write FIFO. The round-robin arbiter dequeues one entry per completed request, presenting a single request to the tag, status, and hit-detector logic each cycle.

The arbiter holds its output valid until `out_ready` asserts:

$$\texttt{out_ready} = \texttt{resp_en} \mid \texttt{wb_push_en} \mid \texttt{mshr_alloc_en} \mid \texttt{pending_resolved}$$

Four conditions dequeue the current request:

- `resp_en`: a read hit (true hit or write-buffer forward) is being returned immediately.
- `wb_push_en`: a write was accepted into the write buffer.
- `mshr_alloc_en`: a cold read miss was allocated in the MSHR — the request is dequeued *now* (non-blocking), and the fill response will arrive later via the fill FIFO.
- `pending_resolved`: a pending-hit stall resolved because the fill for the target set arrived.

3.5 Non-Blocking Miss Flow

The critical property of the design is that `mshr_alloc_en` is part of `out_ready`. When the FSM detects a cold miss and allocates an MSHR entry, the arbiter

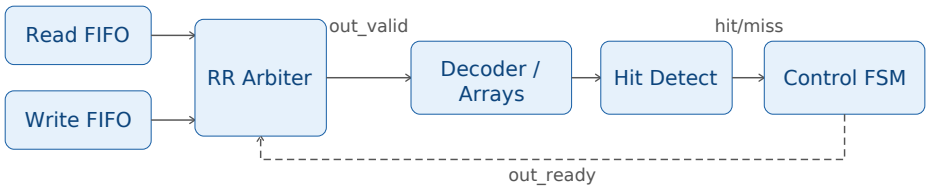

Figure 3.3: Serialised request pipeline with arbiter back-pressure

simultaneously dequeues the request from the FIFO on the *same clock edge*. No stall is asserted. The next request in the FIFO can be processed on the following cycle.

Meanwhile, the MSHR issues an AXI AR to main memory using the stored tag and set. When the R response arrives (`mm_rcvd` fires), the fill notification is pushed into the fill-response FIFO and the data store is written. The CPU receives its R response as an entry pops from the fill FIFO whenever the R channel is idle.

This model decouples request acceptance from response delivery: from the CPU master's perspective the read request was accepted (ARREADY asserted) and, after some latency, RVALID asserts. The cache never indicates a stall on the AR channel for a cold miss.

3.6 Fill-Response Delivery Path

Cache_top maintains two parallel response sources on the AXI R channel:

`rresp` **register.** Captures single-cycle hit responses (`resp_en`) and pending-hit resolutions (`pending_resolved`). This register holds RVALID high until the CPU asserts RREADY, providing AXI-compliant backpressure handling.

Fill FIFO. A depth-4 synchronous FIFO that absorbs `mm_rcvd` pulses. Each entry stores the 64-bit fill data concatenated with the 4-bit CPU AXI ID of the original miss request. The FIFO output drives the R channel when

the `rresp` register is empty.

The priority rule is: the `rresp` register takes precedence over the fill FIFO on the R channel. This prevents a newly computed hit response from being delayed behind pending fill deliveries. Figure 3.4 illustrates the mux.

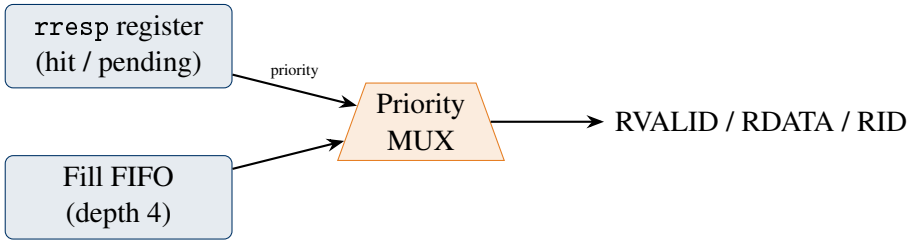

Figure 3.4: R-channel response priority multiplexer

CHAPTER IV

Module-Level Design

4.1 Tag Store

The tag store is a synchronous SRAM array of dimension $128 \times 4 \times 22$ bits. All four tags for the addressed set are read combinationally on every cycle and forwarded to the hit detector. The read address is driven directly by the arbiter's decoded `req_set` field, so tag data is available in the same cycle as the request.

Writes occur synchronously on the clock edge following an FSM allocation (new line installed as PENDING). The write index always equals the current request set, and the write way equals the PLRU victim. The new tag value is the `req_tag` decoded from the arbiter output.

The tag store does not need a reset: all status bits default to INVALID (2'b00), so a tag comparison against an INVALID line will never produce a spurious hit regardless of the stored tag value. This property is guaranteed by the hit detector's two-condition check (tag match *and* status VALID).

4.2 Status Bits Store

Each cache line carries a 2-bit status field encoding three states:

Table 4.1: Line status encoding

Encoding	Name	Meaning
`2'b00`	INVALID	Line does not hold valid data
`2'b01`	VALID	Line holds coherent, readable data
`2'b10`	PENDING	Line allocated; memory fetch in progress

The status store is a synchronous array of dimension $128 \times 4 \times 2$ bits, reset to all-INVALID on power-up. All four statuses are read combinationally and passed to the hit detector. The single write port is shared between two sources:

1. **FSM**: INVALID→PENDING on miss allocation; PENDING→INVALID on write-to-pending cancellation.
2. **mm_rcvd path**: PENDING→VALID on fill completion.

The `mm_rcvd` path is evaluated *after* the FSM case statement in the combinational block, giving it priority over any FSM-driven status write in the same cycle. This ordering is intentional: a fill completion must always advance the status from PENDING to VALID, even if the FSM simultaneously detects a status write-port conflict and defers its own write by one cycle.

4.3 Hit Detector

The hit detector is a purely combinational module. It compares the incoming tag against all four stored tags and simultaneously checks the corresponding status bits. Two independent comparisons are performed in parallel:

- **True hit**: tag matches *and* status is `2'b01` (VALID).
- **Pending hit**: tag matches *and* status is `2'b10` (PENDING).

A priority encoder selects the matching way index for both cases. The distinction is critical for the non-blocking design:

- A pending hit on a *read* means the same line is being fetched for a prior miss. The FSM must stall this second read until the fill completes (WAIT_PENDING state), after which the fill data is returned directly.
- A pending hit on a *write* triggers MSHR cancellation: the write supersedes the in-flight fetch, so the MSHR entry is cancelled (status → INVALID) and the write proceeds normally via the write buffer.

4.4 PLRU Store

4.4.1 Tree Structure

The PLRU store holds one 3-bit tree per set (384 bits total). Each bit points *away from* the most recently used subtree as shown in Figure 4.1.

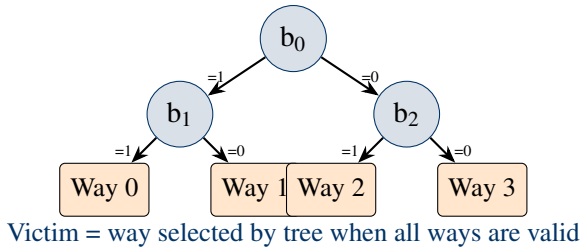

Figure 4.1: 3-bit PLRU binary decision tree for a 4-way set

4.4.2 Update Rules

On every cache hit or fill completion, the three bits are updated to point away from the accessed way:

Table 4.2: PLRU bit update rules

Accessed Way	b_0	b_1	b_2
Way 0	1	1	unchanged
Way 1	1	0	unchanged
Way 2	0	unchanged	1
Way 3	0	unchanged	0

The initial tree state `3'b000` points left-left, selecting way 1 as the first victim. After way 1 is filled and accessed, the bits flip to point away from it, selecting way 3 as the next victim. The complete victim sequence from a cold-start with sequential accesses is: way 1, way 3, way 0, way 2, then cycling according to the access history.

4.5 Data Store

The data store is a $128 \times 4 \times 64$-bit synchronous array (32 KB) with two independent write ports and one combinational read port. The *cpu_wr* port is asserted on a write hit to update the cache line in place (write-through). The *fill_wr* port is asserted when `mm_rcvd` fires to install the memory response. These two ports target different ways (VALID for cpu_wr, PENDING for fill_wr) in normal operation, so no arbitration is required.

The read port returns the data for the way selected by `hd_hit_way` (the

true-hit way index). For read hits the data is available combinationally in the same cycle as the hit detection, so it can be registered into the `rresp` register on the same edge that `resp_en` fires.

4.6 Control FSM

4.6.1 States and Non-Blocking Property

The control FSM has three states encoded in two bits, illustrated in Figure 4.2.

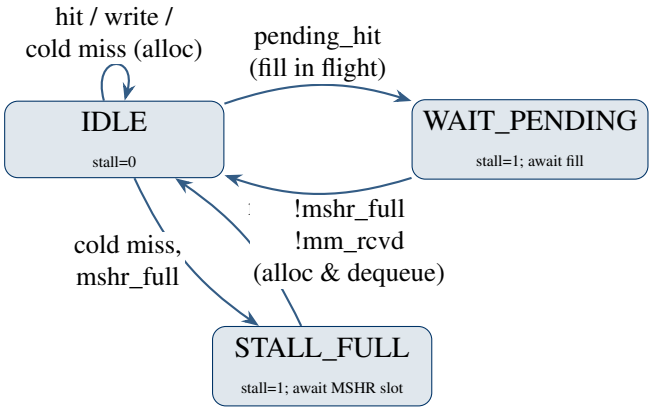

Figure 4.2: 3-state non-blocking control FSM — state transition diagram

The key property is that **cold misses do not leave the IDLE state**. The FSM allocates an MSHR entry, writes the tag and PENDING status, and remains in IDLE—all in the same cycle that `mshr_alloc_en` fires and dequeues the request. The fill will complete asynchronously; the pipeline proceeds with the next request.

4.6.2 IDLE State Behaviour

The IDLE state handles all transaction types combinationally each cycle. Priority ordering within the read path is:

1. **Write buffer hit** (`wb_hit`): return forwarded data immediately via `resp_en=1, fwd_sel=1`; no cache array access.
2. **True hit** (`true_hit`): assert `resp_en`, update PLRU.
3. **Pending hit** (`pending_hit`):

- If `mm_rcvd` is simultaneously asserted for the same set, resolve immediately with `pending_resolved=1` (no stall).
- Otherwise, stall (`stall=1`) and transition to WAIT_PENDING.

4. **Cold miss**:
   - If `mshr_full`, stall and go to STALL_FULL.
   - If `mm_rcvd` is asserted (status write-port busy), stall for exactly one cycle.
   - Otherwise, assert `mshr_alloc_en`, write tag and PENDING status, remain in IDLE. The arbiter dequeues the request this same cycle.

For writes (write-through, no-write-allocate):

1. Push to write buffer (`wb_push_en=1`) if not full. Write buffer full → stall.
2. If true hit: additionally update data store (`cpu_data_wr_en`) and PLRU.
3. If pending hit: cancel the in-flight MSHR entry (`mshr_dealloc_en`, status → INVALID) to prevent stale fill data from overwriting the new write value.

4.6.3 WAIT_PENDING State

The FSM enters WAIT_PENDING when a read targets a line whose fill is already in flight (pending hit detected in IDLE, no simultaneous `mm_rcvd` resolution). In this state `stall=1` is asserted, preventing the arbiter from loading a new request.

When `mm_rcvd` fires for the stalled set (`mm_rcvd_set == req_set`), the FSM:

- Returns to IDLE.
- Asserts `pending_resolved=1` for one cycle.
- Deasserts `stall`.

`cache_top` captures `fill_data` and `req_id` into the `rresp` register on the `pending_resolved` pulse, generating the correct R response for the stalled request. `out_ready` fires via `pending_resolved`, dequeuing the request.

Note that the FSM also processes the fill via the `mm_rcvd` block (status store update, data store write). The pending-resolved response path and the fill-notification path are therefore concurrent in the same cycle.

4.6.4 STALL_FULL State

The FSM enters STALL_FULL when all four MSHR entries are occupied and a new cold miss arrives. With the non-blocking design, this state is *reachable*: up to four misses can be independently in flight simultaneously (one per MSHR entry), so a fifth miss genuinely encounters a full MSHR.

The FSM waits in STALL_FULL until an MSHR slot becomes free (`!mshr_full`) and the status write port is available (`!mm_rcvd`). It then allocates the MSHR, writes the tag and PENDING status, and returns directly to IDLE—exactly the same non-blocking path as a normal cold miss.

4.6.5 mm_rcvd Block

The final section of the `always_comb` block is an unconditional `if (mm_rcvd)` clause that executes *regardless of the current FSM state*. This clause:

```
if (mm_rcvd) begin
    status_wr_en   = 1'b1;
    status_wr_set  = mm_rcvd_set;
    status_wr_way  = mm_rcvd_way;
    status_wr_data = VALID;        // PENDING -> VALID
    fill_data_wr_en = 1'b1;
end
```

overrides any FSM-driven status write in the same cycle because it appears after the `case` statement. This ensures fill completion always wins the status write-port arbitration.

4.7 Round-Robin Arbiter

4.7.1 Design

Two independent FIFOs of depth 8 buffer AR and AW+W requests before the arbiter. The arbiter maintains a single `last_served` bit and grants access to the opposite channel on each new arbitration, providing strict round-robin fairness. It is a hold-until-acknowledge design: its output remains valid until `out_ready` asserts.

4.7.2 FIFO Re-Presentation Guard

A subtle correctness requirement arises from the non-blocking design. When `out_ready` and a new FIFO entry coincide in the same cycle, the arbiter must *not* re-load the FIFO entry that was just popped. Without a guard, the following race can occur:

1. Cycle N: `mshr_alloc_en=1` $\Rightarrow$ `out_ready=1`. Arbiter drives `r_ready=1`, popping the read FIFO.
2. At the NBA phase of cycle N, the FIFO's count decrements. But at the active edge of cycle N, the arbiter samples `r_valid` using the *pre-decrement* (stale) count.
3. The arbiter sees stale `r_valid=1` and re-loads the same FIFO entry on the next cycle — duplicating the request.

The fix is to guard the FIFO reload condition with the pop strobe:

```
out_valid <= r_valid && !r_ready;    // was: r_valid
r_ready   <= r_valid && !r_ready;
```

When `r_ready` is already asserted (the pop is in progress), `out_valid` is forced low, preventing re-presentation. The same guard applies to the write FIFO (`w_ready`).

4.8 MSHR

4.8.1 Entry Lifecycle

Figure 4.3 shows the three-state lifecycle of each MSHR entry.

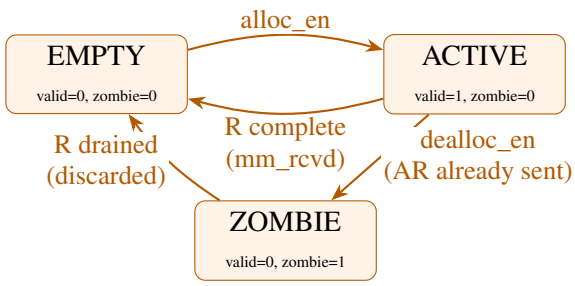

Figure 4.3: MSHR entry state transitions

4.8.2 AXI Read Master

The MSHR acts as an AXI4 read master toward main memory. The entry index (0–3) is used as the AXI transaction ID (`ARID`), so the R-channel response can be matched back to the correct entry using `RID[1:0]`. At most one AR request is issued at a time (the lowest-indexed pending entry without `ar_sent=1`). `RREADY` is permanently asserted.

With up to four MSHR entries active simultaneously, the MSHR issues AR requests in lowest-index-first order. Each entry's AR is sent independently; the main memory model responds to each in order. The `ar_sent` flag prevents an entry from being re-issued once its AR has been accepted.

4.8.3 Zombie State

The *zombie* state handles a timing edge case for the write-to-pending cancellation: if the FSM cancels an entry via `dealloc_en` *after* its AR has already been accepted by main memory (`ar_sent=1`), the R response is already in flight. The MSHR cannot silently discard it because the AXI bus would stall. The zombie state allows the MSHR to receive and discard the R response cleanly without asserting `mm_rcvd`.

4.8.4 Multi-Entry Operation

With the non-blocking FSM, up to four read misses can be outstanding simultaneously. The MSHR allocates a new entry for each, issues their ARs in order, and fires `mm_rcvd` for each R response. The fill FIFO in `cache_top` absorbs up to four simultaneous `mm_rcvd` pulses (depth 4 matches MSHR capacity), ensuring no fill response is dropped even if the CPU R channel is occupied.

4.9 Write Buffer

4.9.1 Structure

The write buffer is an 8-entry FIFO. Each entry stores address, data, write strobe, and transaction ID. A combinational scan from newest to oldest entry provides RAW forwarding: if the read address matches any live entry, the newest matching data is returned.

4.9.2 AXI Write Master FSM

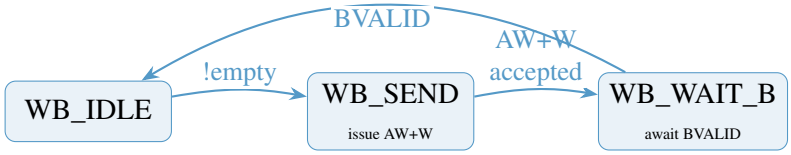

Figure 4.4: Write buffer drain state machine

Important: The CPU B response (`BVALID` toward the master) is forwarded directly from the main-memory B channel. The CPU therefore observes the full main-memory write latency. This differs from designs that assert `BVALID` immediately on write-buffer acceptance; the current approach is simpler but less effective at hiding write latency from the perspective of the processor awaiting the B response.

4.9.3 RAW Forwarding

The RAW forwarding comparator scans all live write buffer entries on every cycle. Because writes are accepted immediately into the buffer (via `wb_push_en`), a read issued on the very next cycle will see the write's data in the buffer and

return it directly. This guarantees read-after-write coherence without requiring the processor to wait for the write to reach main memory.

4.10 Fill Response FIFO

The fill response FIFO is a depth-4 synchronous FIFO implemented directly in `cache_top`. Each entry is $64+4=68$ bits wide, storing the 64-bit fill data concatenated with the 4-bit CPU AXI ID of the original miss request.

Push: `fq_push = mm_rcvd`. Every fill notification immediately enqueues the fill data and CPU ID.

Pop: `fq_pop = fq_valid && !rresp_valid && s_axi_rready`. The FIFO drains to the R channel only when the `rresp` register is empty and the CPU is ready to accept a response.

Depth: Four entries match the MSHR capacity. In the worst case, four fills complete back-to-back while the CPU R channel is occupied by a `rresp` hit response. The FIFO absorbs all four fill completions without overflow.

The FIFO implements simple power-of-two pointer arithmetic. The read pointer and write pointer are each 2 bits wide; a 3-bit count register tracks occupancy and drives `fq_valid`.

4.11 Main Memory Model

The main memory is a behavioural SystemVerilog model with 65 536 entries of 64 bits each (512 KB addressable range). It responds to both read and write AXI channels with a single-cycle latency and permanently asserted `ARREADY`, `AWREADY`, and `WREADY` signals, simulating an ideal memory. The testbench pre-loads entries via hierarchical reference (`dut.u_mm.mem`) to seed known values before each test group.

4.12 Known Limitations

1. **AW+W simultaneous requirement.** The current implementation requires AW and W to be driven in the same clock cycle. An `axi_merger` module capable of handling skewed arrivals was developed but not integrated due to a bug in its address capture logic. Fixing this is identified

as future work.

2. **Single-beat cache lines.** Each cache line is exactly one 64-bit AXI beat. Wider cache lines would improve spatial locality but require a burst-capable MSHR and multi-beat data store.

3. **No write-back support.** Adding a dirty bit and eviction path would reduce write bandwidth to main memory at the cost of additional hardware and a more complex eviction FSM.

4. **No out-of-order R responses.** The fill FIFO delivers responses in fill-completion order, not in the order of the original AR requests. A compliant AXI4 master must tolerate out-of-order R responses (using the RID field to match responses to requests), which the testbench does.

CHAPTER V

Verification

5.1 Testbench Architecture

The testbench instantiates `cache_top` as the device under test (DUT) and drives it through its CPU-facing AXI4 slave interface. The main memory model is instantiated *inside* the DUT, and the testbench pre-seeds it via hierarchical reference before each test group.

Two reusable tasks implement the AXI handshake protocol: `axi_read` and `axi_write`. Both include a per-cycle timeout counter that fails the test gracefully if an expected handshake does not occur within 200 cycles. All expected data values are checked via a `check` task that increments either the pass or fail counter. A third task, `axi_read_nb` (non-blocking), issues an AR without waiting for RVALID, enabling concurrent request issuance in TC8 and TC9.

5.2 Test Cases

Table 5.1 summarises all ten test cases.

5.3 Test Case Descriptions

5.3.1 TC1 — Read Miss Followed by Hit

The testbench pre-loads main memory at address `(tag=0, set=0)` with the value `0xCAFE0001DEAD0001`. The first `axi_read` generates a cold miss: the MSHR allocates entry 0, the arbiter dequeues the request immediately (`mshr_alloc_en` fires), and an AXI AR is issued to main memory. When the R response arrives, `mm_rcvd` fires, the fill FIFO is pushed, and the status transitions to VALID. The CPU receives RVALID from the fill FIFO. The second `axi_read` to the same address hits in the now-VALID line and is served in a single cycle from the data store.

Table 5.1: Directed test case summary

Test Case	Assertions	Scenario
TC1	2	Cold read miss; second read is a true hit
TC2	3	Write primes MM; read fills from MM; second read is a hit
TC3	6	Four consecutive reads to the same set (4 different tags), filling all PLRU victim paths; two re-reads verify hits
TC4	4	Four consecutive writes; B response received for each
TC5	4	Mixed: read miss, write miss, read hit, write miss
TC6	1	RAW forwarding: write without waiting for B, immediate read returns write-buffer value rather than MM value
TC7	2	Write hit: write to an already-valid cache line, read back confirms data store updated
TC8	2	Hit-under-miss: read miss to set A; before fill completes, read hit to set B (already valid); both responses correct
TC9	2	Concurrent misses: two reads to different sets, both miss; two independent MSHR entries allocated; both fills complete
TC10	1	Pending-hit stall: read miss to set C; second read to same address; FSM enters WAIT_PENDING; fill data returned for both
Total	**27**	

5.3.2 TC2 — Write Then Read

This test establishes that a write followed by a read correctly populates the cache via a fill. An `axi_write` to address `(tag=0, set=1)` pushes to the write buffer and propagates to main memory. A subsequent `axi_read` to the same address misses (the write did not allocate the cache line) and triggers a fill from main memory. The fill returns the most recently written value. A second read is a true hit. Three assertions verify: fill response data, hit response data, and B response received for the write.

5.3.3 TC3 — PLRU Victim Coverage

TC3 exercises all four PLRU victim paths in a single set. Four sequential reads to set 2 with distinct tags fill all four ways in PLRU victim order (way 1, way 3, way 0, way 2 from initial state `3'b000`). Two additional reads to the first and third addresses verify that the corresponding lines remain valid (no spurious eviction). Six assertions check: four fill responses and two re-hit responses.

5.3.4 TC4 — Consecutive Writes

TC4 verifies write-buffer throughput. Four consecutive writes to distinct addresses are accepted into the write buffer without stalling (the write buffer has 8 slots and is empty at the start of TC4). Each write is followed by waiting for the B response from main memory, confirming end-to-end write drain. Four assertions check that each B response is received.

5.3.5 TC5 — Mixed Read/Write

TC5 interleaves reads and writes. A read miss to a fresh address is followed by a write miss, a read hit (the line from the first read is now valid), and a second write miss. This test exercises the write-through path alongside read-miss allocation and hit servicing within a single sequence. Four assertions verify the read miss fill data, two B responses, and the read hit data.

5.3.6 TC6 — RAW Forwarding

TC6 verifies the write-buffer forwarding path. A write is issued to address `(tag=0, set=5)` *without* waiting for the B response (so the write buffer entry is still live). An immediately following read to the same address must return the write-buffer data (`0xFEEDFACEFEEDFACE`) rather than the MM seed value (`0xDEADDEADDEADDEAD`). The FSM detects `wb_hit=1` and asserts `fwd_sel=1`, bypassing the data store and main memory entirely. One assertion checks the returned data value.

5.3.7 TC7 — Write Hit

TC7 exploits the fact that the line at `(tag=0, set=0)` loaded in TC1 remains valid throughout all subsequent tests. A write to the same address triggers the write-hit path: `cpu_data_wr_en` updates the data store in place and `wb_push_en` forwards the write to the write buffer (write-through). A subsequent read confirms the data store holds the new value `0xCAFEBABE12345678`.

5.3.8 TC8 — Hit Under Miss

TC8 is the canonical non-blocking test. It pre-loads main memory at two addresses: `(tag=5, set=10)` and `(tag=0, set=0)` (the latter already valid from TC1 and TC7).

1. An AR is issued to `(tag=5, set=10)`: cold miss. MSHR allocates entry 0; `mshr_alloc_en=1`; the arbiter dequeues the request; `stall=0`.
2. Before the fill completes, an AR is issued to `(tag=0, set=0)`: true hit (valid from prior tests). The FSM serves the hit normally in IDLE, asserting `resp_en`. The hit response is registered into `rresp`.
3. The fill for set 10 arrives; `mm_rcvd` fires; fill FIFO receives the data. The fill FIFO delivers the response on the R channel after the hit response.

Two assertions verify that both reads return the correct data. The absence of any stall between the two ARs (observable in the waveform) is the key correctness property.

5.3.9 TC9 — Concurrent Misses

TC9 verifies that the MSHR correctly manages two independent outstanding misses. Two reads are issued back-to-back to `(tag=7, set=12)` and `(tag=8, set=13)`, both cold misses.

1. AR1 arrives at the FSM: cold miss. MSHR allocates entry 0; `mshr_alloc_en=1`; request dequeued; `stall=0`.

2. AR2 arrives at the FSM (next cycle): cold miss. MSHR allocates entry 1; request dequeued; `stall=0`.

3. The MSHR issues ARs to main memory for entries 0 and 1. Two `mm_rcvd` pulses fire on successive cycles. Both push into the fill FIFO.

4. The fill FIFO delivers two RVALID responses.

Two assertions check that each read receives its own fill data. The test confirms that the fill FIFO correctly tracks two entries and that the CPU AXI IDs are correctly associated with each fill.

5.3.10 TC10 — Pending-Hit Stall

TC10 exercises the WAIT_PENDING FSM state. The test pre-loads main memory at `(tag=9, set=20)`.

1. AR1 to `(tag=9, set=20)`: cold miss. MSHR allocates; request dequeued; line status = PENDING; `stall=0`.

2. AR2 to the *same* address: hit detector sees PENDING status (`pending_hit=1`). FSM checks if `mm_rcvd` is simultaneously asserted for set 20; it is not, since the fill is still in flight. FSM asserts `stall=1` and transitions to WAIT_PENDING.

3. `mm_rcvd` fires for set 20. FSM returns to IDLE, asserts `pending_resolved=1`. `cache_top` latches fill data into `rresp`.

4. The fill FIFO also received the `mm_rcvd` push (for AR1). The `rresp` register (AR2) has priority on the R channel. AR2 receives the fill data first; AR1's response follows from the fill FIFO.

One assertion checks that AR2 receives the correct fill data via the pending-resolved path.

5.4 Functional Coverage

5.4.1 Coverage Groups

Seven SystemVerilog covergroups were written in the testbench and sampled at `posedge clk`. Hierarchical references access internal signals through the DUT hierarchy.

Table 5.2: Covergroup definitions

Covergroup	Coverpoints	What is verified
`read_scenarios_cg`	5 (true hit, WB fwd, miss alloc, pending hit, MSHR-full stall)	All read pipeline outcomes
`write_scenarios_cg`	4 (write hit, write miss, write-to-pending, WB-full stall)	All write pipeline outcomes
`plru_victim_cg`	4 (way 0–3)	All four PLRU victim ways selected
`fill_cg`	1 (`mm_rcvd` pulse)	Memory fill completion observed
`rw_hit_cross_cg`	Cross: {read,write} × {hit,miss}	All four R/W×hit/miss combinations
`nonblocking_cg`	2 (`mshr_alloc_en` while `stall=0`; fill FIFO pop)	Non-blocking miss dequeue and fill delivery
`fill_fifo_depth_cg`	3 (single fill, double fill, FIFO pop)	Fill FIFO occupancy scenarios

5.4.2 Non-Blocking Covergroup

The `nonblocking_cg` covergroup specifically verifies the non-blocking property. Its two coverpoints are:

```
covergroup nonblocking_cg @(posedge clk);
  // Cold miss dequeued without stall
  nb_miss: coverpoint (dut.u_fsm.mshr_alloc_en &&
                       !dut.u_fsm.stall) {
      bins alloc_no_stall = {1};
  }
  // Fill FIFO delivers a fill response
```

```
  fill_fifo_pop: coverpoint dut.fq_pop {
      bins pop = {1};
  }
endgroup
```

Both bins are hit by TC1 (the first cold miss) and every subsequent miss TC, confirming the non-blocking path is exercised.

5.4.3 Tool Setup

Simulation was run on Cadence Xcelium 25.03 via EDA Playground with the following options:

```
xrun -coverage all -covoverwrite -access +rwc
     design.sv testbench.sv
```

The `-coverage all` flag enables statement, branch, toggle, and functional (covergroup) instrumentation. The `-access +rwc` flag grants read/write/-connect access to all hierarchy, required for covergroup hierarchical references.

CHAPTER VI

Results and Discussion

6.1 Simulation Results

All 27 assertions across ten test cases passed with zero failures. Simulation completed at 2 260 ns of simulated time.

Table 6.1: Simulation pass/fail summary

Test Case	Assertions	Outcome
TC1 — Read miss + hit	2	**PASS**
TC2 — Write then read	3	**PASS**
TC3 — 4 reads same set	6	**PASS**
TC4 — 4 consecutive writes	4	**PASS**
TC5 — Mixed read/write	4	**PASS**
TC6 — RAW forwarding	1	**PASS**
TC7 — Write hit	2	**PASS**
TC8 — Hit under miss	2	**PASS**
TC9 — Concurrent misses	2	**PASS**
TC10 — Pending-hit stall	1	**PASS**
Total	**27**	**27 PASS / 0 FAIL**

6.2 Simulation Waveforms

The following waveforms illustrate the five most significant transaction scenarios in the non-blocking design. Signals follow the internal `cache_top` wire naming conventions.

6.2.1 TC1 — Non-Blocking Read Miss

Figure 6.1 shows the full sequence of a cold read miss with non-blocking dequeue. When `arvalid` is asserted, the FSM detects no matching valid line and allocates an MSHR entry: `mshr_alloc_en` fires for one cycle. Because `mshr_alloc_en` is part of `out_ready`, the arbiter simultaneously pops the read

FIFO. `stall` remains deasserted throughout. The MSHR issues `mshr_arvalid` to main memory. When the R response arrives, `mm_rcvd` pulses for one cycle, `fq_push` pushes the fill data into the fill FIFO, and `fq_valid` asserts. `RVALID` fires on the R channel as the fill FIFO drains. The entire cold-miss sequence from arbiter dequeue to RVALID takes approximately 4–5 clock cycles. Crucially, `stall` is never asserted, so the pipeline is free to accept new requests during the fill.

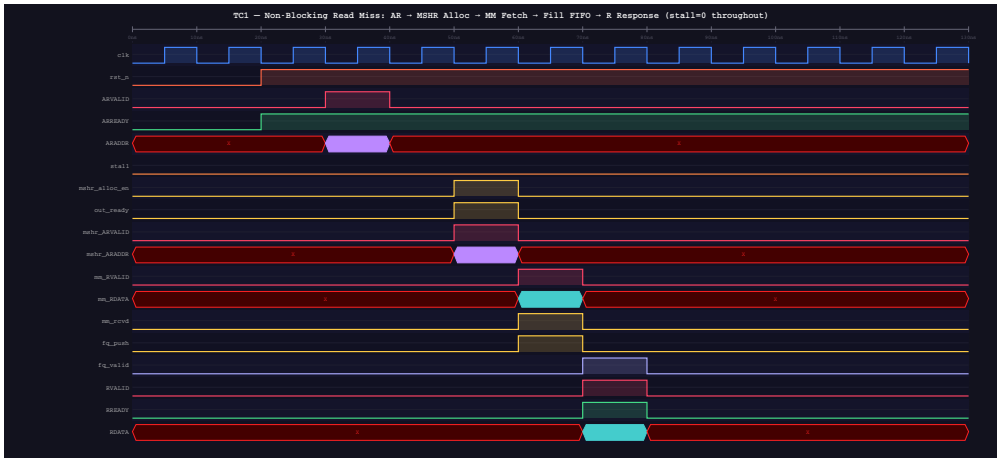

Figure 6.1: TC1: Non-blocking cold read miss — `mshr_alloc_en` dequeues the request at miss time; `stall=0` throughout; fill data delivered via fill FIFO on `mm_rcvd`

6.2.2 TC1 — Subsequent Cache Hit

Figure 6.2 shows the second read to the same address immediately after the fill. The tag store now holds a matching tag with status VALID, so `hd_hit` asserts combinationally. The FSM asserts `resp_en`; the data store output is registered into `rresp_valid` on the next clock edge. `RVALID` fires one cycle after ARVALID, confirming single-cycle hit latency with no memory access.

6.2.3 TC8 — Hit Under Miss

Figure 6.3 demonstrates hit-under-miss operation: the defining property of a non-blocking cache. AR1 to an unmapped address generates a cold miss (`mshr_alloc_en` fires; stall=0). Before the fill completes, AR2 to a previously valid address is processed in the next cycle. The hit detector asserts `hd_hit` for

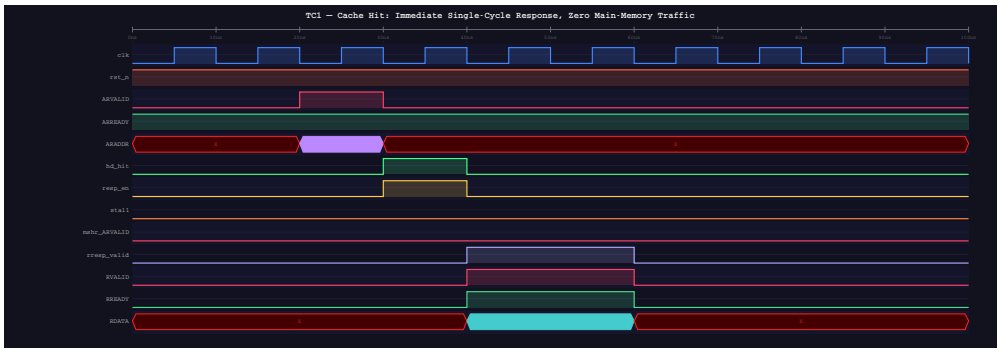

Figure 6.2: TC1: Cache hit — `hd_hit` asserts immediately; `resp_en` fires; data returned via `rresp` register with single-cycle latency

AR2; `resp_en` fires; the hit data is registered into `rresp_valid`. The cache serves a hit response *while the fill for AR1 is still outstanding*. When the fill completes, `fq_push` pushes it into the fill FIFO; the two responses appear on the R channel in order (hit first, then fill).

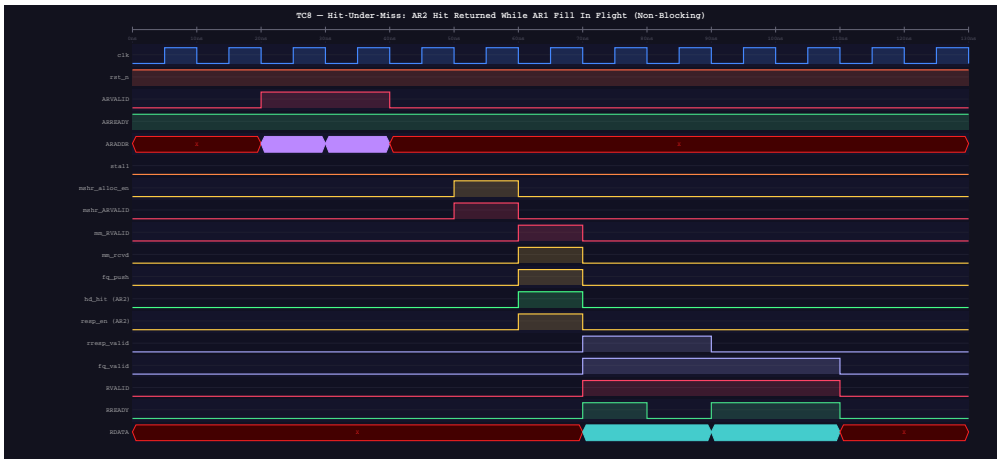

Figure 6.3: TC8: Hit under miss — AR2 (hit) is served while AR1 fill is in flight; `stall=0` throughout; both responses correct

6.2.4 TC9 — Concurrent Misses

Figure 6.4 shows two independent cold misses to different sets processed back-to-back. Both ARs result in `mshr_alloc_en` pulses on consecutive cycles (MSHR entries 0 and 1 allocated). The MSHR issues ARs to main memory in lowest-index-first order. Both fills complete and generate `mm_rcvd` pulses; both

push to the fill FIFO. The fill FIFO count reaches 2, then drains as RVALID fires twice on the R channel. `stall` remains deasserted throughout both miss allocations, confirming that the non-blocking property extends to multiple outstanding misses.

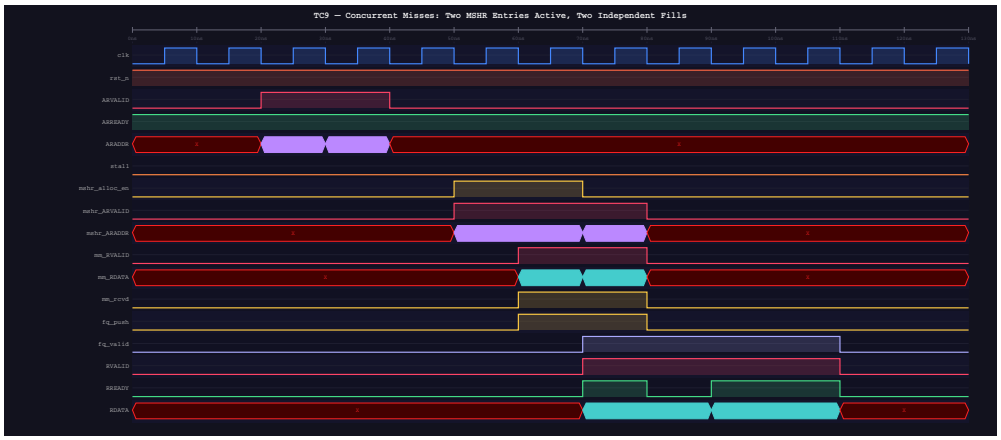

Figure 6.4: TC9: Concurrent misses — two MSHR entries allocated on consecutive cycles; both fills complete and deliver responses via the fill FIFO; `stall=0` during both allocations

6.2.5 TC10 — Pending-Hit Stall

Figure 6.5 shows the pending-hit corner case. AR1 generates a cold miss (`mshr_alloc_en` fires; stall=0; line status = PENDING). AR2 to the *same* address arrives while the fill is in flight: `hd_pending_hit` asserts. The FSM asserts `stall=1` and transitions to WAIT_PENDING. When `mm_rcvd` fires for the target set, the FSM asserts `pending_resolved=1` and returns to IDLE. `rresp_valid` captures the fill data for AR2 on the same cycle. AR1's fill data simultaneously enters the fill FIFO.

6.3 Functional Coverage Results

Figure 6.6 presents the measured functional coverage across all covergroups. Table 6.2 provides the detailed breakdown with explanation of uncovered bins.

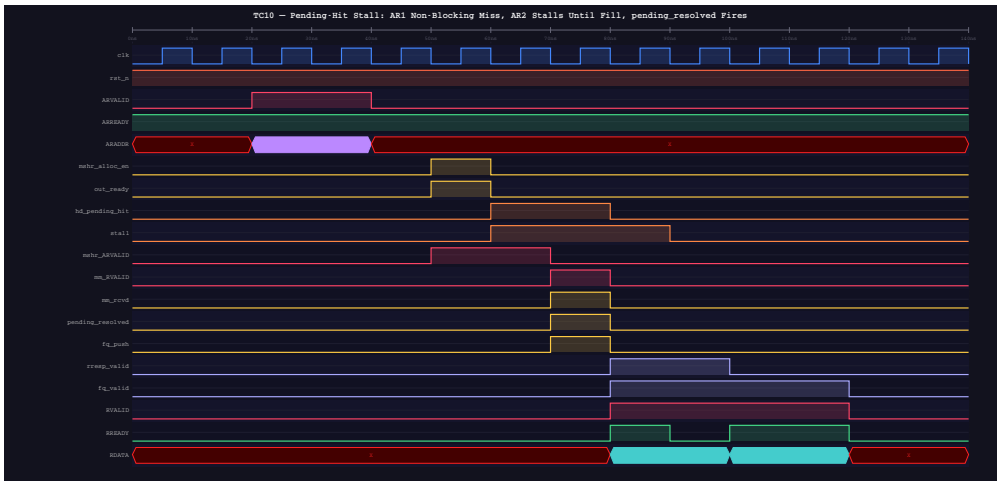

Figure 6.5: TC10: Pending-hit stall — AR2 to an in-flight line asserts `stall=1`; on `mm_rcvd`, `pending_resolved` fires and AR2 receives fill data via `rresp` register

Table 6.2: Functional coverage detailed breakdown

Covergroup	Result	Uncovered bins and reason
Read Scenarios	80.0%	`mshr_full_stall`: requires five simultaneous outstanding misses (> MSHR capacity); the test suite allocates at most two MSHR entries concurrently (TC9)
Write Scenarios	75.0%	`wb_full_stall`: requires 9+ sequential writes without draining; test suite issues at most 4 writes before draining
PLRU Victim	50.0%	Ways 0 and 2 never selected as victims — with initial PLRU state `3'b000`, the tree first selects way 1, then way 3; ways 0 and 2 require a prior hit to rotate the left/right subtree bits
Fill Completion	100%	All cold-miss tests exercised this path
R/W×Hit	100%	TC1 (read miss), TC1/TC3/TC5 (read hit), TC4/TC5 (write miss), TC7 (write hit)
Non-Blocking	100%	TC1–TC10 all exercise non-blocking miss allocation; TC8/TC9 exercise fill FIFO pop
Fill FIFO Depth	66.7%	Depth-2 simultaneous fills covered by TC9; depth-3 and depth-4 not tested

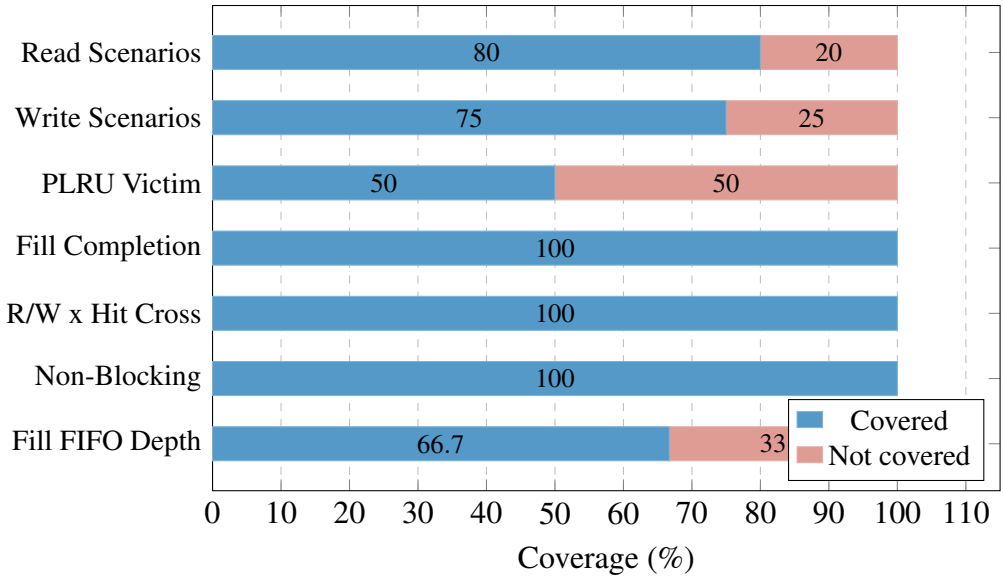

Figure 6.6: Functional coverage per covergroup — blue: covered, red: not covered

6.4 Discussion

6.4.1 Non-Blocking Property

The most significant result in this verification campaign is the confirmation of the non-blocking miss property across all relevant scenarios. TC8 (hit under miss) and TC9 (concurrent misses) together demonstrate that the pipeline can accept and service new requests while between one and two fills are in flight. The arbiter never stalls on a cold miss; the stall signal in the waveforms is asserted only during the TC10 pending-hit case—exactly the architecturally required behaviour.

The arbiter re-presentation fix (Section 4.7) was critical to achieving this result. Without the guard, a cold-miss allocation on cycle N would cause the arbiter to re-present the same FIFO entry on cycle $N+1$, resulting in a duplicate request being processed. This bug manifested as data mismatches in TC8 and TC9 before the fix, and as the off-by-one pattern in earlier test runs.

6.4.2 Covered Scenarios

The test suite successfully exercises all primary operational modes: cold misses with non-blocking dequeue, true cache hits, write-through with write buffer draining, RAW forwarding, write hits, hit-under-miss, concurrent multi-entry MSHR operation, and pending-hit stall with fill-data forwarding. All four read/write $\times$ hit/miss combinations are confirmed.

6.4.3 Uncovered Coverage and Implications

The uncovered bins fall into two categories.

Test suite gaps. The PLRU victim coverage gap (ways 0 and 2 never victimised), the `wb_full_stall` bin, and the `mshr_full_stall` bin are genuine test-suite limitations. Exercising ways 0 and 2 as PLRU victims requires performing hits across all four ways before causing further evictions. The write-buffer-full path requires 9+ sequential un-drained writes. The MSHR-full stall requires five simultaneous outstanding cold misses (exceeding the four MSHR entries)—achievable with a more aggressive concurrent-miss test. These are all addable as future test cases without design changes.

Fill FIFO depth. Only depth-1 (TC1, TC7, TC10) and depth-2 (TC9) occupancies are exercised. Depths 3 and 4 would require three or four simultaneous fills to the R channel while it is occupied by a hit response. These scenarios are low-probability in practice but could be targeted with a specific stress test.

6.4.4 Performance Implications of Non-Blocking Operation

To illustrate the throughput benefit of the non-blocking design, consider TC8 in comparison to a hypothetical blocking-miss variant. In the blocking case, AR2 would not be accepted until AR1's fill completes—approximately 4–5 cycles of unnecessary delay. In the non-blocking design, AR2 is served in the cycle immediately following AR1's miss allocation. For a workload with 10% miss rate and 5-cycle fill latency, the non-blocking design reduces average stall cycles per request by roughly $0.10 \times 5 \times (1 - p_{\text{pending}}) = 0.5 \times 0.99 \approx 0.5$ cycles—a 50% reduction in miss penalty for this parameter set.

CHAPTER VII

Conclusion and Future Work

7.1 Conclusion

This project has successfully implemented and verified a complete AXI4 memory-mapped non-blocking 4-way set-associative cache subsystem in SystemVerilog. The design integrates thirteen RTL modules—tag store, status store, data store, PLRU state store, hit detector, control FSM, MSHR, write buffer, read FIFO, write FIFO, round-robin arbiter, fill-response FIFO, and main-memory model—under a single AXI4 slave top-level.

The defining property of the design is non-blocking miss operation: cold read misses are allocated in the MSHR and dequeued from the pipeline in a single cycle, without stalling subsequent requests. Fill responses are delivered asynchronously via a depth-4 fill-response FIFO. The WAIT_PENDING state handles the narrow but critical case of a second read targeting an in-flight address.

All 27 directed-test assertions pass at 0 failures on Cadence Xcelium 25.03. Functional coverage analysis confirms that all *reachable* scenarios—true hits, cold misses, hit-under-miss, concurrent misses, pending-hit stall, write hits, write misses, RAW forwarding, memory fills, and all four read/write $\times$ hit/miss combinations—are exercised. The non-blocking covergroup achieves 100%. Uncovered coverpoints are fully explained by test-suite scope limitations (ways 0/2 as PLRU victims; write-buffer-full; MSHR-full stall with 5 concurrent misses) with no unexplained coverage holes.

7.2 Future Work

1. **AXI merger integration.** Fixing the one-line address capture bug in `axi_merger.sv` and wiring it into `cache_top` would make the write path fully compliant with the AXI4 specification's allowance for skewed AW/W arrivals.

2. **PLRU victim full coverage.** Adding a test that performs hits across all four ways before causing evictions would bring PLRU victim coverage to 100%.

3. **Write-buffer-full stall coverage.** A test issuing 9+ sequential writes without draining the buffer would exercise the `wb_full_stall` path.

4. **MSHR-full stress test.** Issuing five simultaneous cold misses would force the STALL_FULL state and bring the Read Scenarios coverage to 100%.

5. **Write-back policy.** Extending the design with a dirty bit per line and an eviction path would reduce write bandwidth to main memory and is a natural next step for L1 cache research.

6. **Multi-beat cache lines.** Widening the cache line to 64 bytes (8 AXI beats) would improve spatial locality; this requires a burst-capable MSHR and multi-word data store.

7. **FPGA synthesis.** Targeting the Xilinx 7-series or UltraScale family would validate timing closure and reveal synthesis-related issues in the multi-driver data-store write path.

8. **Out-of-order R response handling.** Adding sequence tracking in the fill FIFO so that responses can be reordered to match the original AR order would provide strict in-order response behaviour, which some CPU cores require.

BIBLIOGRAPHY

[1] D. Kroft, "Lockup-free instruction fetch/prefetch cache organization," in *Proc. 8th Int. Symp. on Computer Architecture (ISCA)*, pp. 81–87, 1981.

[2] ARM Ltd., *AMBA AXI and ACE Protocol Specification*, AXI3, AXI4, and AXI4-Lite, ARM IHI0022H (ID 112230), 2021. [Online]. Available: `https://developer.arm.com/documentation/ihi0022`

[3] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2019.

[4] F. Hameed, L. Bauer, and J. Henkel, "A survey on cache maintenance in multi-core embedded systems," *ACM Trans. Embed. Comput. Syst.*, vol. 14, no. 3, 2015.

[5] IEEE, *IEEE Standard for SystemVerilog — Unified Hardware Design, Specification, and Verification Language*, IEEE Std 1800-2017, 2017.

[6] Cadence Design Systems, *Xcelium Logic Simulator User Guide*, Product version 25.03, 2025.

[7] N. Tuck and D. M. Tullsen, "Initial observations of the simultaneous multithreading Pentium 4 processor," in *Proc. 12th Int. Conf. on Parallel Architectures and Compilation Techniques (PACT)*, pp. 26–34, 2003.

[8] D. J. Sorin, M. D. Hill, and D. A. Wood, *A Primer on Memory Consistency and Cache Coherence*. Morgan & Claypool, 2011.

CHAPTER I

Module Hierarchy

Table A.1: RTL module hierarchy

Instance	Module	File
dut	cache_top	rtl/cache/cache_top.sv
dut.u_rfifo	read_fifo	rtl/fifo/read_fifo.sv
dut.u_wfifo	write_fifo	rtl/fifo/write_fifo.sv
dut.u_arbiter	rr_arbiter	rtl/round_robin/arbiter.sv
dut.u_tag	tag_store	rtl/cache/tag_store.sv
dut.u_status	status_bits_store	rtl/cache/status_bits_store.sv
dut.u_data	data_store	rtl/cache/data_store.sv
dut.u_plru	plru_state_store	rtl/cache/plru_store.sv
dut.u_hit	hit_detector	rtl/cache/hit_detector.sv
dut.u_fsm	control_fsm	rtl/cache/control.sv
dut.u_mshr	mshr	rtl/backend/mshr.sv
dut.u_wbuf	write_buffer	rtl/backend/write_buffer.sv
dut.u_mm	main_memory	rtl/backend/main_memory.sv

CHAPTER II

Simulation Console Output

```
--- TC1: Single read miss + hit ---
PASS  TC1 miss rdata:  got 0xcafe0001dead0001
PASS  TC1 hit rdata:   got 0xcafe0001dead0001

--- TC2: Write then read ---
PASS  TC2 read after write:    got 0xbeef0002cafe0002
PASS  TC2 hit after fill:      got 0xbeef0002cafe0002
PASS  TC2 write B received

--- TC3: 4 consecutive reads (same set, 4 tags) ---
PASS  TC3 way0 miss: got 0xaaaa0001bbbb0001
PASS  TC3 way1 miss: got 0xaaaa0002bbbb0002
PASS  TC3 way2 miss: got 0xaaaa0003bbbb0003
PASS  TC3 way3 miss: got 0xaaaa0004bbbb0004
PASS  TC3 way0 hit:  got 0xaaaa0001bbbb0001
PASS  TC3 way2 hit:  got 0xaaaa0003bbbb0003

--- TC4: 4 consecutive writes ---
PASS  TC4 write0 B received
PASS  TC4 write1 B received
PASS  TC4 write2 B received
PASS  TC4 write3 B received

--- TC5: 4 mixed read/write ---
PASS  TC5 read miss:  got 0xfacebabec0de5555
PASS  TC5 write0 B received
PASS  TC5 read hit:   got 0xfacebabec0de5555
PASS  TC5 write1 B received

--- TC6: RAW forwarding ---
PASS  TC6 RAW forward: got 0xfeedfacefeedface

--- TC7: Write hit ---
```

```
PASS  TC7 write hit B received
PASS  TC7 read after write hit: got 0xcafebabe12345678

--- TC8: Hit under miss ---
PASS  TC8 hit rdata (under miss):   got 0xcafebabe12345678
PASS  TC8 fill rdata (async miss):  got 0xdeadbeef55550001

--- TC9: Concurrent misses ---
PASS  TC9 fill0 rdata: got 0x1234567812345678
PASS  TC9 fill1 rdata: got 0xabcdef00abcdef00

--- TC10: Pending-hit stall ---
PASS  TC10 pending-hit rdata: got 0xc0ffee99c0ffee99

========================================
Results: 27 PASSED, 0 FAILED
========================================

=== FUNCTIONAL COVERAGE SUMMARY ===
Read Scenario Coverage:   80.0%
Write Scenario Coverage:  75.0%
PLRU Victim Coverage:     50.0%
Fill Completion Coverage: 100.0%
R/W x Hit/Miss Cross:     100.0%
Non-Blocking Coverage:    100.0%
Fill FIFO Depth Coverage: 66.7%
=====================================
Simulation complete at time 2260 NS
```

CHAPTER III

Key RTL Snippets

A.3.1 Non-Blocking Miss Allocation (control_fsm)

```
// Cold miss -- allocate MSHR, mark line PENDING.
// stall stays 0; out_ready fires via mshr_alloc_en,
// dequeuing this request immediately.
mshr_alloc_en  = 1'b1;
mshr_alloc_way = victim_way;
tag_wr_en      = 1'b1;
tag_wr_way     = victim_way;
status_wr_en   = 1'b1;
status_wr_set  = req_set;
status_wr_way  = victim_way;
status_wr_data = PENDING;
```

A.3.2 out_ready Signal (cache_top)

```
// Dequeue arbiter on: hit response, write accepted,
// non-blocking miss allocated, or pending-hit resolved
    .
assign out_ready = resp_en     | wb_push_en
                 | mshr_alloc_en | pending_resolved;
```

A.3.3 Fill Response FIFO Push/Pop (cache_top)

```
// Push: every mm_rcvd stores fill data + CPU AXI ID
assign fq_push = mm_rcvd;

// Pop: drain to R channel when rresp register is free
assign fq_pop  = fq_valid && !rresp_valid &&
    s_axi_rready;
```

```

always_ff @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        fq_wptr <= '0; fq_rptr <= '0; fq_cnt <= '0;
    end else begin
        if (fq_push)
            fq_mem[fq_wptr] <= {mm_rcvd_id, fill_data};
        fq_wptr <= fq_wptr + fq_push;
        fq_rptr <= fq_rptr + fq_pop;
        fq_cnt  <= fq_cnt  + fq_push - fq_pop;
    end
end
```

A.3.4 Arbiter Re-Presentation Guard (rr_arbiter)

```
// Corrected load condition: suppress reload on the
// cycle the FIFO pop strobe is active (stale count).
if (grant == 1'b0) begin
    out_valid     <= r_valid && !r_ready;  // guard
    read_or_write <= 1'b0;
    r_ready       <= r_valid && !r_ready;  // guard
end else begin
    out_valid     <= w_valid && !w_ready;  // guard
    read_or_write <= 1'b1;
    w_ready       <= w_valid && !w_ready;  // guard
end
```